Inheritance in Java

A Student is a Person

CSC241 • BAI • Fall 2026

Dr Muhammad Shahid Bhatti
COMSATS University Islamabad, Lahore Campus

Introductory lecture • Java 17 examples

What you will be able to do

Skill	Evidence of understanding
Model	Explain why Student IS-A Person
Implement	Use extends and initialize inherited state
Trace	Predict constructor and method output
Diagnose	Repair access and constructor errors
Design	Choose inheritance or composition

One example throughout the lecture

Stage	Question
1 Relationship	Why should Student extend Person?
2 Construction	How does a Student get a name?
3 Behavior	How can Student change introduce()?
4 Reasoning	Which calls compile and which body runs?

Two classes repeat the same idea

Person	Student
name	name
getName()	getName()
introduce()	introduce()
	regNo

A change to shared behavior would require two edits.

The IS-A test

Statement	Relationship
A Student is a Person	IS-A: inheritance can fit
A Student has an Address	HAS-A: composition can fit
A Car has an Engine	HAS-A: composition can fit
A Car is an Engine	Incorrect model

Shared code alone does not establish an IS-A relationship.

Superclass and subclass

Term	In our example
Superclass / parent / base class	Person: the more general type
Subclass / child / derived class	Student: the more specific type
Inherited behavior	getName(), when accessible
Specialization	regNo and student-specific behavior

The first inheritance declaration

```
class Student extends Person {
    private String regNo;
}

Student s = new Student();
```

extends names the direct superclass.

This tiny example assumes Person has an accessible no-argument constructor.

What a subclass receives

Element	What happens?
Accessible fields and methods	May be inherited
Private superclass members	Not inherited by the subclass
Constructors	Never inherited
Superclass private state	Still exists in a subclass object

Access and object state are different questions.

Person owns its name

```
class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Person initializes its private state.

Other classes use getName() to read it.

Student initializes both parts

```
class Student extends Person {
    private String regNo;

    public Student(String name,
                   String regNo) {
        super(name);
        this.regNo = regNo;
    }
}
```

super(name) calls the Person constructor.

this.regNo belongs to the current Student object.

One Student object

Student s = new Student("Ayesha", "FA26-BAI-007");

State within s	Value	Initialized by
Person.name (private)	Ayesha	Person constructor
Student.regNo (private)	FA26-BAI-007	Student constructor

The constructor chain initializes one object.

Using inherited behavior

```
Student s = new Student(
    "Ayesha", "FA26-BAI-007");

System.out.println(s.getName());
```

Output

```
Ayesha
```

Student does not redeclare getName().

Constructor order: predict first

```
class Person {
    Person() {
        System.out.println("Person");
    }
}
class Student extends Person {
    Student() {
        System.out.println("Student");
    }
}
```

new Student();

Which line appears first?

Why does Person run at all?

Constructor bodies run parent first

Step	What happens
1	new Student() begins the constructor chain
2	Student() implicitly invokes super()
3	Person() prints Person
4	Student() prints Student

Output: Person
Student

Why this constructor fails

```
class Person {
    Person(String name) { }
}

class Student extends Person {
    Student() { }
}
```

Compile-time error

The implicit super() needs Person().

Person declares only Person(String).

this(...) and super(...)

```
Student(String name) {
    this(name, "Pending");
}

Student(String name, String regNo) {
    super(name);
    this.regNo = regNo;
}
```

this(...) delegates to another constructor in Student.

super(...) delegates to Person.

Java 17: use either as the first statement, never both directly.

Private data stays protected by its class

```
class Student extends Person {
    public void showName() {
        // System.out.println(name);
        System.out.println(getName());
    }
}
```

Direct access to Person.name fails.

The public inherited getter works.

Access in a hierarchy

Modifier on superclass member	Subclass access
public	Accessible
protected	Accessible, with cross-package restrictions
No modifier (package access)	Accessible when in the same package
private	No direct access

Keep fields private. Expose the operations that subclasses need.

Checkpoint: three quick decisions

Question	Your answer
Does Student inherit Person(String)?	
Can Student directly read private Person.name?	
Can s.getName() work without redeclaration?	

Give the reason, not just yes or no.

Overriding specializes behavior

```
// In Person
public void introduce() {
    System.out.println("I am " + getName());
}

// In Student
@Override
public void introduce() {
    System.out.println(
        "Student: " + getName());
}
```

Same method name and parameter types.

Student supplies a new body.

@Override helps the compiler catch mistakes.

super.method() reuses the parent behavior

```
// Inside Student
@Override
public void introduce() {
    super.introduce();
    System.out.println("Reg: " + regNo);
}
```

For Ayesha:

I am Ayesha
Reg: FA26-BAI-007

Overloading versus overriding

	Overloading	Overriding
Parameters	Different parameter list	Same parameter types
Inheritance needed?	No	Yes
Meaning	Another method variant	Specialized instance behavior
Example	show() and show(int)	Student replaces introduce()

An overriding checklist

Check	Rule for today
Signature	Same name and parameter types
Return	Use the same return type in our examples
Visibility	Do not reduce access
Restrictions	final methods cannot be overridden
Method kind	Static methods are hidden, not overridden

A Person reference can refer to a Student

```
Person p = new Student(
    "Ayesha", "FA26-BAI-007");

p.introduce();
```

Reference type: Person

Actual object type: Student

The overriding Student body runs.

Two decisions govern a method call

Decision	What determines it?
Is the call allowed?	Declared reference type and access rules
Which overriding instance body runs?	Actual object type at runtime

```
Person p = new Student("Ayesha", "007");
p.introduce();   // allowed, Student body
// p.study();    // not a Person method
```

Inheritance patterns in Java

Pattern	Example	Allowed for classes?
Single	Student extends Person	Yes
Multilevel	GraduateStudent extends Student	Yes
Hierarchical	Student and Teacher extend Person	Yes
Multiple direct superclasses	C extends A, B	No

Every class except Object has one direct superclass.

Inheritance versus composition

Design question	Model	Java declaration
A Student is a Person	Inheritance	class Student extends Person
A Student has an Address	Composition	private Address address;
An Engine is part of a Car	Composition	private Engine engine;

A sensible design can use both relationships.

Live coding: add a Teacher

Extend the existing Person class with Teacher.

1. Add a private String subject field.
2. Initialize name using super(name).
3. Override introduce() to print name and subject.
4. Create a Teacher and call introduce().

Output challenge

```
class A {
    A() { System.out.println("A"); }
    public void show() {
        System.out.println("A.show");
    }
}
class B extends A {
    B() { System.out.println("B"); }
    @Override public void show() {
        System.out.println("B.show");
    }
}
```

```
A x = new B();
x.show();
```

Write all output lines in order.

Repair the design and the code

Problem	Proposed repair
class Car extends Engine	
Student reads private Person.name	
Student() omits a required super argument	
@Override void show(int n) replaces show()	

Key takeaways

Concept	Remember
IS-A	A subclass is a more specific superclass type
extends	Creates a direct superclass relationship
super(...)	Initializes the superclass part of one object
private	Preserves control of state within its class
Overriding	Specializes inherited instance behavior
Dispatch	Object type selects the overriding body

Exit questions and further reading

Why can a Student call getName() but not read name?

Why does Person construction happen before the Student body?

When would you use a contained object instead of extends?

Reading: Oracle Java Tutorials, “Inheritance”
https://docs.oracle.com/javase/tutorial/java/IandI/subclasses.html
Next: deeper polymorphism and object relationships